DevOps Day 20: Local App & Test Verification Guide

Target System: Ubuntu / WSL
Objective: Complete Task 3 verification workflow (Application -> Local Run -> Working -> Tests -> PASS) prior to Dockerization.

1. High-Level Workflow Meaning

Verifying your Python program directly on the host computer ensures all business logic, frameworks, and syntax configurations work perfectly prior to containment. This isolates code errors from environment errors: if it fails inside Docker later, you know the code is healthy and only the container wrapper requires configuration refinement.

2. Required Project Directory Structure

Your folder must match this strict structural blueprint to guarantee clean module resolution:

```
devops-day20-docker/
■■■ src/
■ ■■■ app.py                # Main application logic
■■■ tests/
■ ■■■ test_app.py           # Automated test suite
■■■ requirements.txt          # Project dependencies
■■■ README.md                 # Documentation
■■■ .gitignore                # Excluded version control entries
```

3. Step-by-Step Technical Execution

Step A: Clear Environment & Target Permissions

Navigate out of systemic, read-only structures (like /home) and target your personal space directory.

```
cd ~
mkdir -p devops-day20-docker/src devops-day20-docker/tests
cd devops-day20-docker
```

Step B: Construct Application Code Base

Write the core Python script utilizing the Flask micro-framework. Port 8080 is implemented to prevent system binding permission conflicts.

```
cat << 'EOF' > src/app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
return "Hello, DevOps Day 20!"

if __name__ == '__main__':
```

```
app.run(host='0.0.0.0', port=8080)
EOF
```

Step C: Construct Automated Testing Suite

Write your structural tests targeting application assertion logic and verification parameters.

```
cat << 'EOF' > tests/test_app.py
import pytest
from src.app import app

@pytest.fixture
def client():
with app.test_client() as client:
yield client

def test_home_page(client):
response = client.get('/')
assert response.status_code == 200
assert b"Hello, DevOps Day 20!" in response.data
EOF
```

Step D: Define Explicit Dependencies

Document exact semantic framework versions to ensure immutable configuration builds.

```
cat << 'EOF' > requirements.txt
flask==3.0.3
pytest==8.2.2
EOF
```

Step E: Environment Initialization & Local Run

Configure system core tools, isolate dependencies using a virtual environment, and initiate your app.

```
sudo apt update && sudo apt install python3 python3-pip python3-venv -y
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python3 src/app.py
```

Step F: Test Suite Execution

Open a secondary standalone terminal prompt window, execute environment runtime pathways, and query validation tools.

```
cd ~/devops-day20-docker
source venv/bin/activate
pytest
```